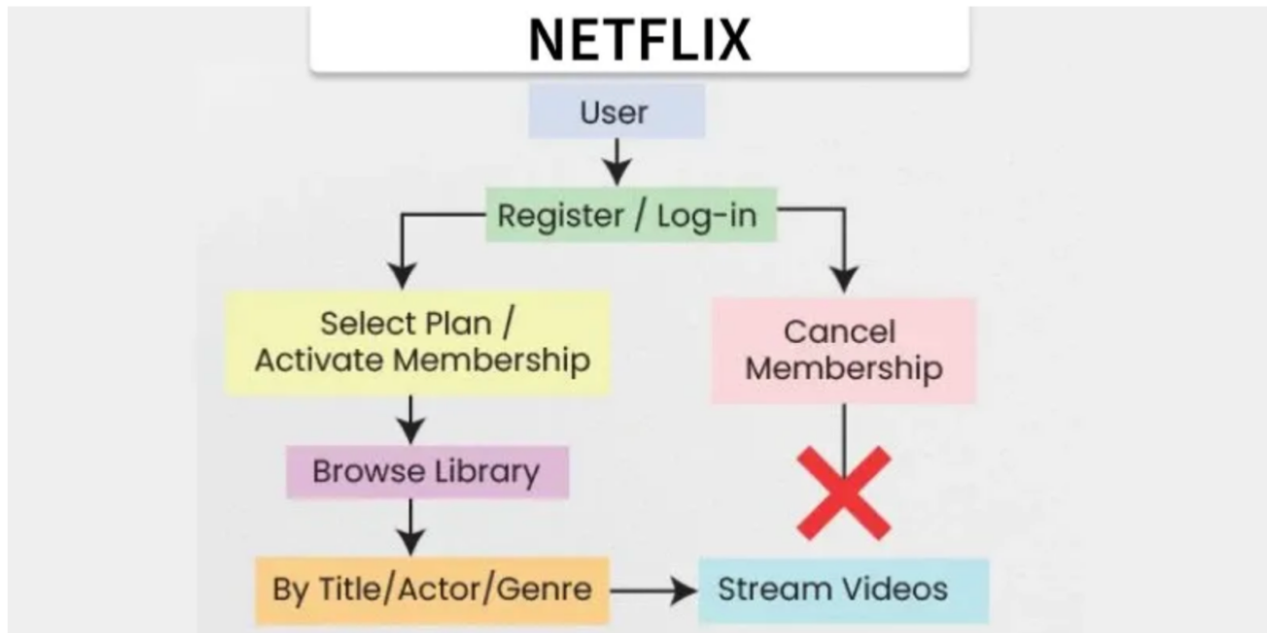


Exploring System Design of Netflix

DAY 122
09/Nov/25

Designing Netflix is a quite common question of system design rounds in interviews. In the world of streaming services, Netflix stands as a monopoly, captivating millions of viewers worldwide with its vast library of content delivered seamlessly to screens of all sizes. Behind this seemingly effortless experience lies a nicely crafted system design. In this article, we will study Netflix's system design.



Requirement of Netflix System Design

- Users should be able to create accounts, log in, and log out.
- Subscription management for users.
- Allow users to play videos and pause, play, rewind, and fast-forward functionalities.
- Ability to download content for offline viewing.
- Personalized content recommendations based on user preferences and viewing history.

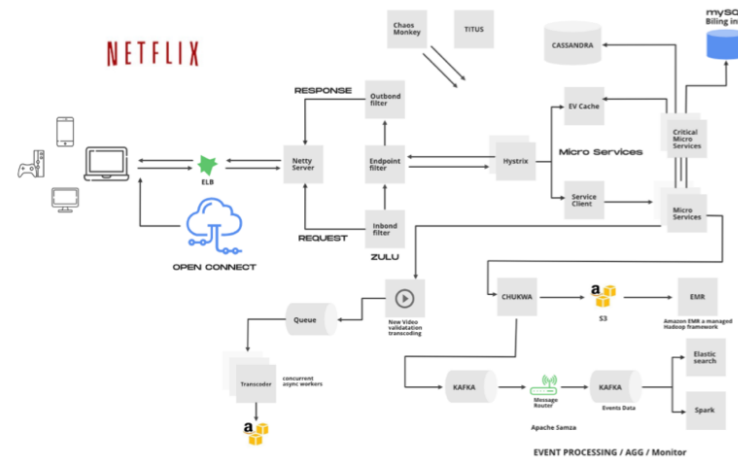
1.2. Non-Functional Requirements

- Low latency and high responsiveness during content playback.
- Scalability to handle a large number of concurrent users.
- High availability with minimal downtime.
- Secure user authentication and authorization.
- Intuitive user interface for easy navigation.

Highlevel Design of Netflix System Design

We all are familiar with Netflix services. It handles large categories of movies and television

content and users pay the monthly rent to access these contents. Netflix has **180M+** subscribers in **200+** countries.



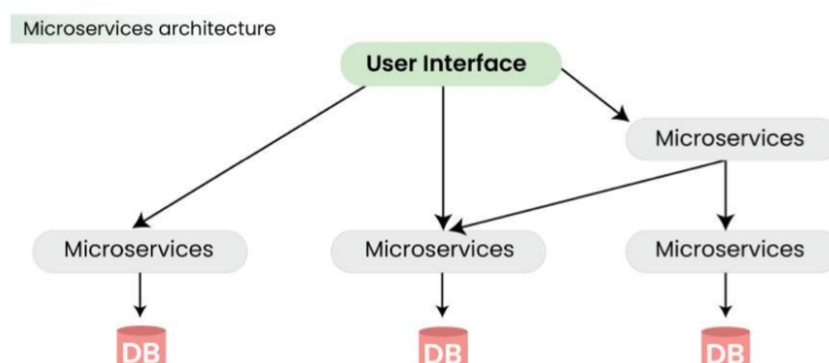
Netflix works on two clouds **AWS** and **Open Connect**. These two clouds work together as the backbone of Netflix and both are highly responsible for providing the best video to the subscribers.

The application has mainly 3 components:

- **Client (User Device):**
TV, Xbox, laptop, or mobile phone used to browse and play Netflix videos.
- **OC (Open Connect / Netflix CDN):**
Netflix's global CDN delivers videos from the nearest server for faster streaming, reducing latency and load on central servers.
- **Backend (Database & Services):**
Manages non-streaming tasks like content onboarding, video processing, distribution to servers, and traffic management, mostly powered by AWS.

Microservices Architecture of Netflix

Netflix's architectural style is built as a collection of services. This is known as microservices architecture and this power all of the APIs needed for applications and Web apps. When the request arrives at the endpoint it calls the other microservices for required data and these microservices can also request the data from different microservices. After that, a complete response for the API request is sent back to the endpoint.



In a Microservice architecture, Services should be independent of each other. For example. The Video Storage Service would be decoupled from the Service responsible for transcoding videos.

How to make microservice architecture reliable?

- **Use Hystrix:**

Helps isolate failures and prevent cascading issues across services.

- **Separate Critical Microservices:**

Keep essential features (search, navigation, play) independent or reliant only on reliable services, ensuring high availability even in failures.

- **Stateless Servers:**

Design servers to be replaceable—if one fails, traffic is redirected to another without dependency on any single server.

3. Capacity Estimation (Order-of-Magnitude)

3.1 Concurrency & Sessions

- Assume **Daily Active Users (DAU)** $\approx$ 250 million.
- **Peak concurrency** $\sim$ 5–10% $\Rightarrow$ 12.5–25 million simultaneous streams (use 15–20 million for quick math).
- **Sessions per user per day** $\approx$ 2 $\Rightarrow$ 500 million play starts/day $\rightarrow$ $\sim$ 5.8k Queries per Second (QPS) for play starts on average (allow bursts 4–5 $\times$).

3.2 Bitrate & Egress (Adaptive Bitrate, ABR)

- Streams use an ABR ladder (e.g., 240p–4K); assume $\sim$ 3–4 Megabits per second (Mbps) average across users.
- Rule of thumb: 1 million concurrent at 3 Mbps $\approx$ 3 Terabits per second (Tbps) $\approx$ 375 GB/s.
- Example at 20 million concurrent and 3.65 Mbps avg $\Rightarrow$ $\sim$ 73 Tbps ($\sim$ 9.1 TB/s) edge egress.

3.3 Edge vs Origin (Open Connect impact)

- Netflix's private Content Delivery Network (CDN), Open Connect, localizes traffic on Open Connect Appliances (OCAs) inside/near ISPs.
- With $\sim$ 98% cache hit-rate, origin egress is only $\sim$ 2% of edge: from 73 Tbps edge $\rightarrow$ $\sim$ 1.46 Tbps origin.
- Benefits: lower startup time, fewer rebuffers, reduced backbone cost.

3.4 Control-Plane Load (Browse. Search. Personalization)

3.4 Control Plane Load (Control, Security, Authentication)

- A busy hour might see ~50 million active users. If each issues ~15 API calls over ~10 minutes:
 - ~750 million calls/10 min $\Rightarrow$ ~1.25 million Requests per Second (RPS) burst across services.
- Keep EVCache/Redis hit-rates >95–99% so databases see a small fraction of that load.

3.5 Event/Telemetry Ingest

- Per session, assume ~150 events (playback Quality of Experience, UI actions).
- 500 million sessions/day $\times$ 150 = ~75 billion events/day.
- At ~500 bytes/event, that's ~37.5 TB/day (before compression).
- Size Kafka partitions for 2–5 $\times$ peak; mirror to Amazon Simple Storage Service (S3)/Parquet for batch jobs.

3.6 Storage Footprint (Control Plane, not video masters)

- Catalog metadata (titles/seasons/episodes/availability) = hundreds of GB, replicated.
- Artwork variants = multi-TB hot set in object storage + CDN.
- Personalization caches (per-profile home pages) = 100–200 KB each, materialized for minutes.

3.7 Peaks, Regionality, and Safety Buffers

- 3.1 Concurrency & Sessions
- Assume Daily Active Users (DAU) $\approx$ 250 million.

3.8 One Worked Mini-Example (talk track)

- 15 million concurrent @ 3 Mbps $\Rightarrow$ 45 Tbps edge (~5.6 TB/s).
- 98% edge hit $\Rightarrow$ ~0.9 Tbps origin.
- DAU 250 M; 2 sessions $\Rightarrow$ 500 M play starts/day $\Rightarrow$ ~5.8k QPS average; bursts 20–30k QPS at drops.
- Events: 75 B/day $\approx$ ~37 TB/day ingest.
- Control plane: busy-hour aggregate ~1M+ RPS, with stores shielded by caches.

3.9 Why Edges (Open Connect), succinctly

- Performance: shorter paths $\rightarrow$ faster Time To First Frame (TTFF), steadier ABR.
- Cost: shifts egress off backbone/cloud.
- Resilience: regional isolation; quick failover between OCAs.